Name/NetID: Nathan Kariuki / nkari011
Session Date: May 26, 2026 (1 hour)
Objective: Implement ApplyTracker and rewrite loader.ht.

Attempts made:
The tricky part was that updater threads can finish out of order. Thread A might commit version 5 before thread B even starts on version 3. A query at version 6 still needs all of 1 through 5 done before it can run. A plain counter breaks here because it would advance past 3 the moment 4 and 5 finish, even if 3 is still in progress.

My first attempt was just checking if applied_max >= target but that breaks when completions arrive out of order. The fix is buffering early completions in a set and only advancing applied_max when the gap above it closes:

```
void MarkDone(uint64_t v) {
   lock_guard<mutex> lk(mu);
   if (v == applied_max + 1) {
      applied_max++;
      while (done_above.count(applied_max + 1)) {
         done_above.erase(applied_max + 1);
         applied_max++;
      }
      cv.notify_all();
   } else {
      done_above.insert(v);
   }
}
```

For loader.h I replaced the separate adds/delays vectors with a single unified record list. This makes the producer loop in main a simple walk through one list in arrival order, which is exactly what the spec requires.

Solutions/Workarounds:
Version assignment stays in the producer not the loader since both ADD and Q consume version slots but D does not. Keeping that logic in main made it cleaner.

Commits: add ApplyTracker for out-of-order version synchronization;
loader.h: parse ADD, Q, D records with version assignment

Experiments/Analysis:
Compiled and checked that the record list parsed correctly by printing the counts of ADD, Q, and D records from a small test trace. Counts matched what I expected.

Learning outcome:

A gapless counter with an overflow buffer is a clean pattern for out-of-order completion tracking. applied_max only advances when there are no holes below it, so WaitUntil is guaranteed to be correct regardless of thread scheduling.